UNIVERSIDAD TECNOLÓGICA METROPOLITANA

DIVISIÓN TIC

BASES DE DATOS EN LA NUBE

“UTM Desk NoSQL Cloud”
Fase 1: Avances técnicos

UNIDAD II. BASES DE DATOS NO RELACIONALES EN LA NUBE

MONGODB ATLAS, API REST Y POSTMAN

PRESENTAN:
Arroyo León Luis Diego
Can Domínguez Víctor Emmanuel
Montalvo Canché Emir Emanuel
Vargas Guzmán Kevin Aarón

Mérida, Yucatán, 15 de octubre de 2026

UTM Desk NoSQL Cloud: avances técnicos de la Fase 1

UTM Desk NoSQL Cloud es un sistema de gestión, resolución y analítica de incidencias del campus de la Universidad Tecnológica Metropolitana. Centraliza en MongoDB Atlas las incidencias que hoy llegan por correo y hojas de cálculo desde tres fuentes con datos muy distintos: cómputo y telecomunicaciones, infraestructura física y sistemas académicos. Este documento reúne los tres entregables de la Fase 1: la selección justificada de tecnologías, la especificación de requisitos funcionales y no funcionales, y el modelado formal de datos en JSON/BSON.

1. Selección justificada de tecnologías

1.1 Criterios de decisión

Se eligió cada tecnología con cuatro criterios: que cumpla lo que obliga el enunciado, que tenga capa gratuita suficiente para el proyecto, que el equipo pueda explicarla línea por línea en la defensa individual y que se pueda desplegar en la nube sin depender de una computadora del equipo.

Tabla 1

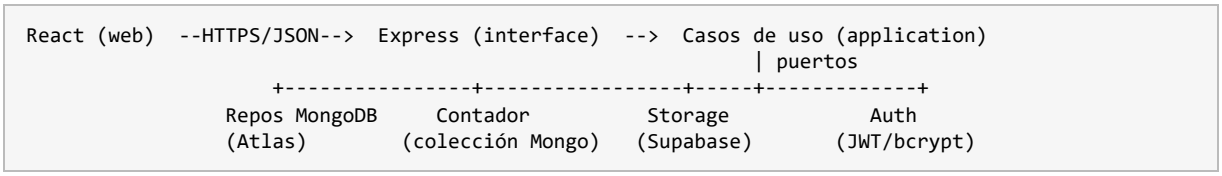
Tecnologías seleccionadas y su justificación

Capa	Elección	Justificación
Base de datos	MongoDB Atlas M0 con el driver nativo `mongodb`	Es el núcleo obligatorio. El driver nativo permite mostrar el validador `$jsonSchema`, los índices y los pipelines tal cual, sin una capa de mapeo que los oculte. Se descarta Mongoose por esa razón.
Backend / API	Node.js 20 con Express (JavaScript, ESM)	Un solo lenguaje para API y UI, soporte nativo de JSON (el mismo formato de los documentos), amplia documentación con MongoDB y despliegue directo en Vercel. HTTP simple con códigos 200, 201, 400, 401, 403, 404 y 409.
Validación	Zod en la aplicación y `$jsonSchema` en Atlas	Dos líneas de defensa: Zod entrega al usuario un error útil; el validador de Atlas protege la colección aunque la aplicación falle.
Interfaz	React, Vite, Tailwind CSS y Recharts	Permite un formulario que cambia según la categoría (el reflejo del polimorfismo), gráficas simples para la analítica y una compilación estática barata de desplegar.
Evidencias	Supabase Storage (bucket privado)	Capa gratuita, URLs firmadas nativas con vigencia corta y bucket privado. Los binarios nunca entran a MongoDB, lo que protege los 512 MB de M0.

Capa	Elección	Justificación
Contador	Colección `contadores` en MongoDB con `$inc` atómico	El docente aceptó que el contador viva en MongoDB. `findOneAndUpdate` con `$inc` y `upsert` es atómico y evita una dependencia más (sin Redis que instalar ni mantener).
Autenticación	JWT, bcrypt y roles	Cumple la precondición de CU-01 (sesión activa validada) y permite permisos por rol aplicados en el servidor.
Pruebas	Vitest, Supertest, Postman y Newman	Postman es entregable obligatorio; Newman ejecuta la misma colección desde la terminal para repetir las pruebas.
Despliegue	Vercel (API y web) y GitHub	Demostración en vivo en la nube sin depender de la laptop. El cliente de MongoDB se cachea en memoria para entornos serverless.

1.2 Arquitectura

La aplicación sigue una arquitectura limpia. El dominio (enumeraciones, transiciones de estado, esquemas Zod y generación de código) no conoce Express, MongoDB ni Supabase; los casos de uso dependen de puertos (interfaces) que la capa de infraestructura implementa. Así cualquier integrante puede explicar un caso de uso sin conocer todo el sistema.



1.3 Decisiones que precisan al enunciado

El enunciado contiene dos ambigüedades que el equipo resolvió de forma explícita:

- **Prioridades.** El texto dice "3 prioridades" pero enumera cuatro (baja, media, alta, crítica). Se implementan cuatro.
- **Código y contador.** CU-01 menciona un consecutivo diario por categoría y un código de ejemplo `INC-2026-0042`. Se guarda el consecutivo diario en `consecutivo_diario` y el código formal usa un consecutivo anual, ambos con `$inc` atómico en la colección `contadores`.
- **Ciclo de vida.** Una incidencia resuelta no se reabre, porque el índice TTL cuenta 60 días desde `fecha_resolucion`. Solo el coordinador la pasa de resuelta a cerrada.

2. Requisitos del sistema

2.1 Actores y permisos

Tabla 2

Roles y permisos del sistema

Acción	Reportante	Técnico	Coordinador	Directivo
Crear incidencia	Sí	Sí	Sí	No
Ver incidencias	Solo las propias	Todas	Todas	Todas
Subir evidencia	Solo en las propias	Sí	Sí	No
Cambiar estado y horas	No	Sí	Sí	No
Gestionar etiquetas	No	Sí	Sí	No
Cerrar, editar campos universales, borrar	No	No	Sí	No
Buscar	Solo las propias	Sí	Sí	Sí
Analítica	No	No	Sí	Sí

2.2 Requisitos funcionales

Tabla 3

Requisitos funcionales (RF)

ID	Requisito	Descripción
RF-01	Registrar incidencia (CU-01)	Cuando un usuario autenticado envía `POST /api/incidencias` con los campos universales y un `detalle_especifico` válido para su categoría, el sistema inserta el documento con estado `abierta`, historial inicial, `tags` y `evidencias_adjuntas` vacíos, `horas_invertidas` en 0 y un código `INC-AAAA-NNNN`, y responde 201. El reportante sale de la sesión, no del cuerpo.
RF-02	Validación del alta	Si falta un campo, un enum es inválido, el `detalle_especifico` no corresponde a la categoría o hay campos extra, el sistema responde 400 con la lista de errores y no inserta nada. Atlas rechaza con `$jsonSchema` lo que Zod deje pasar.
RF-03	Contador	Al registrar una incidencia, el sistema incrementa de forma atómica el contador `contador:incidencias:YYYY-MM-DD:categoria` de la colección `contadores` y guarda el valor en `consecutivo_diario`; también obtiene el consecutivo anual para el código.
RF-04	Cargar evidencia (CU-02)	El dueño de la incidencia, un técnico o un coordinador sube un archivo jpeg, png o pdf de máximo 4 MB. El sistema lo guarda en el bucket, genera una URL firmada temporal y agrega los metadatos con `$push` a `evidencias_adjuntas`. Responde 404 si la incidencia no existe y 400 si el archivo no es válido.

ID	Requisito	Descripción
RF-05	Consultar expediente	`GET /api/incidencias/:codigo` devuelve el documento completo en una sola lectura, con las URLs firmadas regeneradas. Un reportante solo ve las suyas (403 en otro caso).
RF-06	Listar y filtrar	`GET /api/incidencias` con filtros opcionales (categoría, estado, edificio, prioridad), paginación y orden por fecha de creación descendente, apoyado en el índice compuesto.
RF-07	Actualizar estado y bitácora (CU-03)	Un técnico o coordinador envía estado nuevo, horas y comentario. Un único `updateOne` aplica `$set`, `$push` e `$inc`. Si otra persona cambió el estado antes, responde 409 sin modificar nada; si la transición no es válida, 400.
RF-08	Etiquetas (CU-04)	`$addToSet` para agregar (sin duplicados) y `$pull` para quitar. Las etiquetas se normalizan a minúsculas y sin espacios sobrantes.
RF-09	Búsqueda textual (CU-05)	`$text` sobre el índice ponderado en español, con `score` de relevancia y orden descendente. El reportante solo obtiene resultados de sus propias incidencias.
RF-10	Analítica (CU-06)	Pipeline 1 de eficiencia por edificio y Pipeline 2 de ranking de los diez tags más frecuentes, disponibles para coordinador y directivo.
RF-11	Depuración automática (CU-07)	El índice TTL elimina el documento 60 días naturales después de `fecha_resolucion`, que se asienta como `Date` al resolver y no existe en los demás estados.
RF-12	Editar y borrar	El coordinador edita título, descripción, prioridad y ubicación con `$set`, y borra el documento junto con sus archivos del bucket.
RF-13	Sesión y permisos	`POST /api/auth/login` devuelve un JWT. Sin token o con token inválido responde 401; sin permiso por rol, 403.
RF-14	Datos de demostración	Un script crea la colección con validador e índices y carga 24 incidencias sintéticas y los usuarios de demo; otro exporta el JSON final y el script maestro de `mongosh`.
RF-15	Interfaz	La UI permite iniciar sesión, crear incidencias con un formulario por categoría, listar, filtrar, buscar, ver el expediente, actualizar estado y consultar la analítica según el rol.

2.3 Requisitos no funcionales

Tabla 4

Requisitos no funcionales (RNF)

ID	Atributo	Descripción
RNF-01	Rendimiento	El expediente se lee en una sola operación, sin joins ni `$lookup`. Listados y búsqueda usan índice (`IXSCAN` y `TEXT` en `explain()`).
RNF-02	Almacenamiento	La base cabe en los 512 MB de M0 y no guarda binarios.
RNF-03	Seguridad	Contraseñas con bcrypt, sesión con JWT, `helmet`, CORS restringido, límite de peticiones, validación de entrada, bucket privado y secretos solo en variables de entorno.

ID	Atributo	Descripción
RNF-04	Integridad	Validación en dos capas y cambios de estado atómicos y sin condición de carrera.
RNF-05	Disponibilidad	API y UI desplegadas en la nube contra Atlas; la demostración no depende de la computadora del equipo.
RNF-06	Usabilidad	Interfaz en español, usable en móvil, con estados vacíos y mensajes de error claros.
RNF-07	Mantenibilidad	Arquitectura limpia, pruebas automáticas, README y guía de defensa.
RNF-08	Entrega	PDF en APA 7.ª edición, repositorio Git, colección Postman exportada y script maestro.

2.4 Casos límite considerados

- Dos técnicos cambian el estado a la vez: uno recibe 409 y no se pierden horas ni bitácora.
- `detalle_especifico` de otra categoría: 400.
- Etiqueta repetida con distinto formato (`Switch` y `switch`): queda una sola.
- Archivo de 0 bytes, de más de 4 MB o un ejecutable renombrado como imagen: 400, porque se valida el tipo real y no solo la extensión.
- Horas negativas, no numéricas o mayores a 24 por intervención: 400.
- Búsqueda con acentos o plurales (`filtración`, `filtraciones`): el índice en español los trata como la misma raíz.
- Incidencia con más de 60 días resuelta: desaparece sola y la UI maneja el 404.

3. Modelado de datos en JSON/BSON

3.1 Colecciones

La base de datos `utmdesk` tiene dos colecciones. `incidencias_campus` es la principal: cada incidencia es un solo documento que embebe su bitácora, etiquetas y metadatos de evidencias, por lo que el expediente completo se lee con una sola operación. `usuarios` guarda las cuentas y sus roles (`nombre`, `correo` único, `password_hash`, `rol`, `tipo` solo para reportantes y `activo`). La colección `contadores` guarda los consecutivos con la forma `{ _id: "contador:incidencias:2026-10-15:computo_redes", valor: 3 }`.

3.2 Campos universales

Tabla 5

Campos comunes a todas las incidencias

Campo	Tipo BSON	Descripción
`codigo`	string	Código formal `INC-AAAA-NNNN`, único.
`consecutivo_diario`	int	Valor del contador diario por categoría.
`titulo`, `descripcion`	string	Resumen (5 a 120 caracteres) y detalle (10 a 2000).
`categoria`	string (enum)	`computo_redes`, `infraestructura` o `academico`.
`prioridad`	string (enum)	`baja`, `media`, `alta` o `critica`.
`estado`	string (enum)	`abierta`, `en proceso`, `en espera`, `resuelta` o `cerrada`.
`reportante`	object	`usuario_id` (ObjectId), `nombre`, `tipo` (docente, estudiante, administrativo) y `correo`.
`ubicacion`	object	`edificio`, `piso` (opcional) y `espacio`.
`asignado_a`	object	Opcional. Se agrega cuando un técnico actúa por primera vez.
`detalle_especifico`	object	Subdocumento polimórfico según la categoría (sección 3.3).
`tags`	array de string	Etiquetas únicas, de 2 a 40 caracteres.
`historial_seguimiento`	array de object	Bitácora: `fecha`, `autor`, `estado_anterior` (opcional), `estado_nuevo`, `comentario` y `horas`.
`evidencias_adjuntas`	array de object	`nombre_archivo`, `tipo_mime`, `peso_kb`, `storage_path`, `url_storage`, `url_expira` y `fecha_carga`.
`horas_invertidas`	double	Acumulado con `$inc`.
`fecha_creacion`, `fecha_actualizacion`	date	Marcas de tiempo nativas.
`fecha_resolucion`	date	Solo existe en `resuelta` y `cerrada`. Alimenta el índice TTL.

Nota. Ningún documento contiene campos con valor `null` ni vacíos de relleno: lo que no aplica simplemente no existe.

3.3 Heterogeneidad: detalle_especifico por categoría

El subdocumento `detalle_especifico` es el que cambia de forma según la fuente de la incidencia. La Tabla 6 resume las tres variantes y el código de las secciones siguientes muestra un ejemplo completo de cada una.

Tabla 6

Variantes del subdocumento detalle_especifico

Categoría	Campos obligatorios	Campos opcionales
`computo_redes`	`tipo_falla` (switch, access_point, servidor, equipo_laboratorio), `direccion_ip` (IPv4), `vlan` (1 a 4094), `puerto_uplink`, `equipos_afectados` (mínimo 1)	Ninguno.
`infraestructura`	`tipo_dano` (filtracion, luminaria, climatizacion, cerradura, otro), `riesgo_seguridad` (ninguno, bajo, medio, alto), `requiere_contratista` (bool)	`insumos_requeridos` (arreglo de strings) y `contratista_externo` (solo si `requiere_contratista` es verdadero).
`academico`	`plataforma` (lms, portal_calificaciones, correo_institucional), `periodo_escolar` (formato AAAA-1 a AAAA-3), `estudiantes_impactados` (mínimo 1)	`codigo_curso` (solo para lms y portal_calificaciones).

Ejemplo: categoría computo_redes

```
{
  "_id": { "$oid": "6710a1f2c3d4e5f6a7b8c901" },
  "codigo": "INC-2026-0042",
  "consecutivo_diario": 3,
  "titulo": "Switch del laboratorio LC-2 sin respuesta",
  "descripcion": "Desde las 8:00 los equipos del laboratorio no obtienen red.",
  "categoria": "computo_redes",
  "prioridad": "alta",
  "estado": "en proceso",
  "reportante": { "usuario_id": { "$oid": "6710a1f2c3d4e5f6a7b8c001" }, "nombre": "Mtra. Ana Pech",
"tipo": "docente", "correo": "ana.pech@utmetropolitana.edu.mx" },
  "ubicacion": { "edificio": "Laboratorios de Cómputo", "piso": 1, "espacio": "LC-2" },
  "asignado_a": { "usuario_id": { "$oid": "6710a1f2c3d4e5f6a7b8c002" }, "nombre": "Luis Cauich" },
  "detalle_especifico": { "tipo_falla": "switch", "direccion_ip": "10.20.2.1", "vlan": 20,
"puerto_uplink": "Gi1/0/24", "equipos_afectados": 28 },
  "tags": ["switch", "sin-red", "laboratorio"],
  "historial_seguimiento": [
    { "fecha": { "$date": "2026-10-12T14:05:00Z" }, "autor": { "usuario_id": { "$oid":
"6710a1f2c3d4e5f6a7b8c001" }, "nombre": "Mtra. Ana Pech" },
      "estado_nuevo": "abierta", "comentario": "Incidencia registrada", "horas": 0 },
    { "fecha": { "$date": "2026-10-12T15:30:00Z" }, "autor": { "usuario_id": { "$oid":
"6710a1f2c3d4e5f6a7b8c002" }, "nombre": "Luis Cauich" },
      "estado_anterior": "abierta", "estado_nuevo": "en proceso", "comentario": "Revisando uplink",
"horas": 1.5 }
  ],
  "evidencias_adjuntas": [
    { "nombre_archivo": "switch-lc2.jpg", "tipo_mime": "image/jpeg", "peso_kb": 412,
      "storage_path": "INC-2026-0042/switch-lc2.jpg", "url_storage": "https://…firmada",
      "url_expira": { "$date": "2026-10-12T16:30:00Z" }, "fecha_carga": { "$date": "2026-10-
12T15:40:00Z" } }
  ],
  "horas_invertidas": 1.5,
  "fecha_creacion": { "$date": "2026-10-12T14:05:00Z" },
  "fecha_actualizacion": { "$date": "2026-10-12T15:30:00Z" }
}
```

Ejemplo: categoría infraestructura (resuelta)

```
{
  "codigo": "INC-2026-0047", "consecutivo_diario": 1,
  "titulo": "Filtración en el techo del aula A-203",
  "descripcion": "Gotea agua sobre las bancas del fondo cada vez que llueve.",
  "categoria": "infraestructura", "prioridad": "media", "estado": "resuelta",
  "reportante": { "usuario_id": { "$oid": "…" }, "nombre": "Mtro. Carlos May", "tipo": "docente",
"correo": "carlos.may@utmetropolitana.edu.mx" },
  "ubicacion": { "edificio": "Edificio A", "piso": 2, "espacio": "A-203" },
  "detalle_especifico": {
    "tipo_dano": "filtracion", "riesgo_seguridad": "medio", "requiere_contratista": true,
    "contratista_externo": "Impermeabilizaciones del Sureste",
    "insumos_requeridos": ["impermeabilizante acrílico", "sellador de juntas"]
  },
  "tags": ["filtracion", "techo", "recurrente"],
  "historial_seguimiento": [ { "fecha": { "$date": "…" }, "autor": { "…": "…" }, "estado_nuevo":
"abierta", "comentario": "Incidencia registrada", "horas": 0 } ],
  "evidencias_adjuntas": [],
  "horas_invertidas": 6,
  "fecha_creacion": { "$date": "2026-09-20T09:10:00Z" },
  "fecha_actualizacion": { "$date": "2026-10-02T17:00:00Z" },
  "fecha_resolucion": { "$date": "2026-10-02T17:00:00Z" }
}
```

Ejemplo: categoría academico

```
{
  "codigo": "INC-2026-0051", "consecutivo_diario": 2,
  "titulo": "Bloqueo de cuentas de correo institucional",
  "descripcion": "Varios estudiantes no pueden iniciar sesión desde el lunes.",
  "categoria": "academico", "prioridad": "critica", "estado": "abierta",
  "reportante": { "usuario_id": { "$oid": "…" }, "nombre": "Daniela Chan", "tipo": "estudiante",
"correo": "daniela.chan@utmetropolitana.edu.mx" },
  "ubicacion": { "edificio": "Administrativo", "espacio": "Coordinación TIC" },
  "detalle_especifico": { "plataforma": "correo_institucional", "periodo_escolar": "2026-3",
"estudiantes_impactados": 85 },
  "tags": ["correo", "acceso"],
  "historial_seguimiento": [ { "fecha": { "$date": "…" }, "autor": { "…": "…" }, "estado_nuevo":
"abierta", "comentario": "Incidencia registrada", "horas": 0 } ],
  "evidencias_adjuntas": [],
  "horas_invertidas": 0,
  "fecha_creacion": { "$date": "2026-10-14T08:00:00Z" },
  "fecha_actualizacion": { "$date": "2026-10-14T08:00:00Z" }
}
```

Nota. Los valores marcados con "…" se abrevian solo para este documento; en la base real son identificadores y fechas completas.

3.4 Validador $jsonSchema

El validador se crea con `createCollection` y nivel estricto. Aplica el polimorfismo con `oneOf` (cada categoría exige su forma de `detalle_especifico`) y obliga `fecha_resolucion` solo en los estados resuelta y cerrada. Los campos numéricos aceptan `["int","double"]` porque el driver de JavaScript guarda los números como double.

```
db.createCollection("incidencias_campus", {
  validationLevel: "strict", validationAction: "error",
  validator: { $jsonSchema: {
    bsonType: "object",
    required:
["codigo","consecutivo_diario","titulo","descripcion","categoria","prioridad","estado",
               "reportante","ubicacion","detalle_especifico","tags","historial_seguimiento",
               "evidencias_adjuntas","horas_invertidas","fecha_creacion","fecha_actualizacion"],
    additionalProperties: false,
    properties: {
      _id: { bsonType: "objectId" },
      codigo: { bsonType: "string", pattern: "^INC-\\d{4}-\\d{4,}$" },
      consecutivo_diario: { bsonType: ["int","long","double"], minimum: 1 },
      titulo: { bsonType: "string", minLength: 5, maxLength: 120 },
      descripcion: { bsonType: "string", minLength: 10, maxLength: 2000 },
      categoria: { enum: ["computo_redes","infraestructura","academico"] },
      prioridad: { enum: ["baja","media","alta","critica"] },
      estado: { enum: ["abierta","en proceso","en espera","resuelta","cerrada"] },
      reportante: { bsonType: "object", required: ["usuario_id","nombre","tipo","correo"] },
      asignado_a: { bsonType: "object", required: ["usuario_id","nombre"] },
      ubicacion: { bsonType: "object", required: ["edificio","espacio"] },
      detalle_especifico: { bsonType: "object" },
      tags: { bsonType: "array", uniqueItems: true, items: { bsonType: "string", minLength: 2,
maxLength: 40 } },
      historial_seguimiento: { bsonType: "array", minItems: 1 },
      evidencias_adjuntas: { bsonType: "array" },
      horas_invertidas: { bsonType: ["int","double"], minimum: 0 },
      fecha_creacion: { bsonType: "date" },
      fecha_actualizacion: { bsonType: "date" },
      fecha_resolucion: { bsonType: "date" }
    },
    allOf: [
      { oneOf: [   // polimorfismo: cada categoría exige SU detalle_especifico
        { properties: { categoria: { enum: ["computo_redes"] },
            detalle_especifico: { bsonType: "object", additionalProperties: false,
              required: ["tipo_falla","direccion_ip","vlan","puerto_uplink","equipos_afectados"] }
} },
        { properties: { categoria: { enum: ["infraestructura"] },
            detalle_especifico: { bsonType: "object", additionalProperties: false,
              required: ["tipo_dano","riesgo_seguridad","requiere_contratista"] } } },
        { properties: { categoria: { enum: ["academico"] },
            detalle_especifico: { bsonType: "object", additionalProperties: false,
              required: ["plataforma","periodo_escolar","estudiantes_impactados"] } } }
      ] },
      { anyOf: [   // fecha_resolucion obligatoria solo al resolver o cerrar
        { properties: { estado: { enum: ["abierta","en proceso","en espera"] } } },
        { required: ["fecha_resolucion"], properties: { estado: { enum: ["resuelta","cerrada"] } }
}
      ] }
    ]
  } }
})
```

Las reglas condicionales finas (por ejemplo, que `contratista_externo` solo exista cuando `requiere_contratista` es verdadero) las valida Zod, porque `$jsonSchema` no soporta `if/then`.

3.5 Índices

Tabla 7
Índices de la colección incidencias_campus

Nombre	Definición	Propósito
`idx_cat_estado_edificio`	`{ categoria: 1, estado: 1, "ubicacion.edificio": 1 }`	Compuesto obligatorio; sirve a los filtros de la lista.
`idx_ttl_resolucion`	`{ fecha_resolucion: 1 }` con `expireAfterSeconds: 5184000`	TTL obligatorio de 60 días (CU-07).
`idx_texto_es`	`{ titulo: "text", descripcion: "text" }`, `default_language: "spanish"`, pesos 10 y 5	Búsqueda de texto completo ponderada (CU-05).
`idx_codigo`	`{ codigo: 1 }` único	Acceso por código en las rutas.
`idx_reportante`	`{ "reportante.usuario_id": 1, fecha_creacion: -1 }`	Lista "mis incidencias".

3.6 Operadores atómicos

Tabla 8
Operadores atómicos y su uso

Operador	Caso de uso	Uso en el sistema
`$set`	CU-03	Cambia `estado`, `fecha_actualizacion` y, al resolver, `fecha_resolucion`.
`$push`	CU-02, CU-03	Agrega una entrada a `historial_seguimiento` o una evidencia a `evidencias_adjuntas`.
`$inc`	CU-01, CU-03	Acumula `horas_invertidas` e incrementa el contador de la colección `contadores`.
`$addToSet`	CU-04	Agrega una etiqueta sin duplicarla.
`$pull`	CU-04	Retira una etiqueta errónea.

El cambio de estado es una sola operación atómica. El filtro incluye el estado que el servidor leyó, de modo que si otra persona lo cambió antes, `matchedCount` es 0 y el sistema responde 409:

```
db.incidencias_campus.updateOne(
  { codigo, estado: estadoLeido },
  { $set:  { estado: nuevo, fecha_actualizacion: ahora,
             ...(nuevo === "resuelta" ? { fecha_resolucion: ahora } : {}) },
    $push: { historial_seguimiento: { fecha: ahora, autor, estado_anterior: estadoLeido,
                                      estado_nuevo: nuevo, comentario, horas } },
    $inc:  { horas_invertidas: horas } }
)
```

3.7 Pipelines de analítica

```
// Pipeline 1: eficiencia operativa por edificio
db.incidencias_campus.aggregate([
  { $match: { estado: { $in: ["resuelta","cerrada"] }, fecha_resolucion: { $exists: true } } },
  { $addFields: { horas_transcurridas: { $dateDiff: {
      startDate: "$fecha_creacion", endDate: "$fecha_resolucion", unit: "hour" } } } },
  { $group: { _id: "$ubicacion.edificio",
              promedio_horas_resolucion: { $avg: "$horas_transcurridas" },
              total_horas_trabajo: { $sum: "$horas_invertidas" },
              total_incidencias: { $sum: 1 } } },
  { $sort: { promedio_horas_resolucion: -1 } }
])

// Pipeline 2: ranking de los 10 tags más frecuentes
db.incidencias_campus.aggregate([
  { $unwind: "$tags" },
  { $group: { _id: "$tags", total: { $sum: 1 }, edificios: { $addToSet: "$ubicacion.edificio" } }
},
  { $sort: { total: -1, _id: 1 } },
  { $limit: 10 }
])
```

3.8 Estados y transiciones

```
abierta  -->  en proceso  <-->  en espera
   |             |                |
   +-------------+--------+-------+
                          v
                     resuelta --(solo coordinador)--> cerrada
```

Desde `abierta`, `en proceso` o `en espera` se puede pasar a cualquiera de los otros dos y a `resuelta`. Nada se reabre y `cerrada` es un estado final. Al pasar a `cerrada` se conserva la `fecha_resolucion` que la incidencia ya tenía.

3.9 Datos de demostración

El script de carga genera 24 incidencias sintéticas, ocho por categoría, en cinco ubicaciones del campus (Edificio A, Edificio B, Laboratorios de Cómputo, Biblioteca y Administrativo). Incluye 5 abiertas, 4 en proceso, 3 en espera, 6 resueltas y 6 cerradas (12 con `fecha_resolucion`, todas de los últimos 45 días para que el TTL no las borre), al menos 17 con evidencias y fallas recurrentes (tres filtraciones en el aula A-203 y tres fallas de red en el laboratorio LC-2) para que la analítica tenga sentido.

Referencias

Express. (s. f.). *Express: Fast, unopinionated, minimalist web framework for Node.js*. https://expressjs.com/

MongoDB, Inc. (s. f.-a). *Aggregation pipeline*. MongoDB Manual. https://www.mongodb.com/docs/manual/core/aggregation-pipeline/

MongoDB, Inc. (s. f.-b). *Schema validation*. MongoDB Manual. https://www.mongodb.com/docs/manual/core/schema-validation/

MongoDB, Inc. (s. f.-c). *Text indexes*. MongoDB Manual. https://www.mongodb.com/docs/manual/core/indexes/index-types/index-text/

MongoDB, Inc. (s. f.-d). *TTL indexes*. MongoDB Manual. https://www.mongodb.com/docs/manual/core/index-ttl/

Postman, Inc. (s. f.). *Postman documentation*. https://learning.postman.com/docs/

Supabase. (s. f.). *Storage*. https://supabase.com/docs/guides/storage

Vercel. (s. f.). *Vercel documentation*. https://vercel.com/docs

Zod. (s. f.). *Zod: TypeScript-first schema validation*. https://zod.dev/